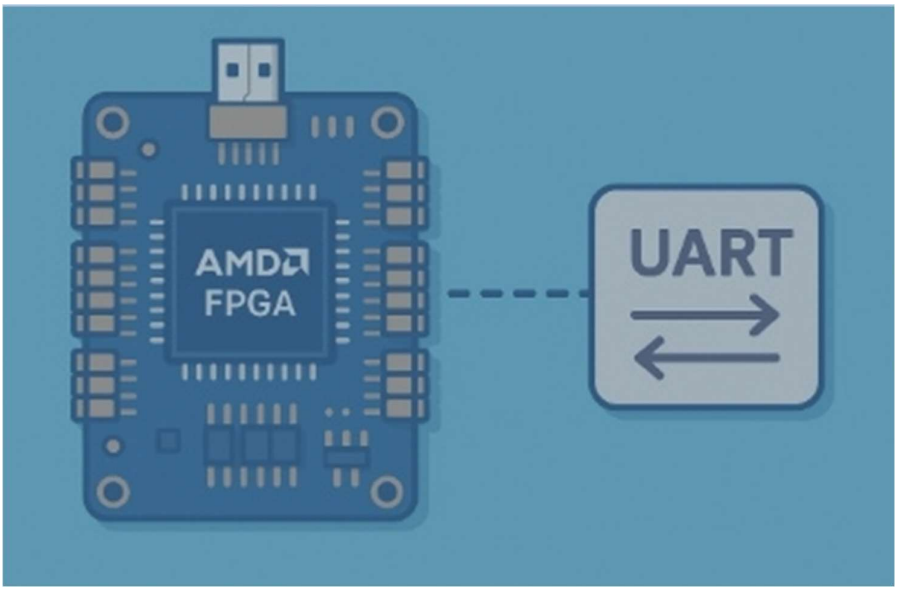
DIGITAL SYSTEMS LAB

UART TRANSMITTER–RECEIVER SYSTEM

Lab 02



OBJECTIVE

Design and verification of an FPGA-based Universal Asynchronous Receiver/Transmitter (UART) featuring a baud-rate controller, an 8-bit transmitter and receiver, parity checking, and framing-error detection.

ANDROMACHE SPYROU – ANASTASIA EFTHYMIOU

04059 – 03988

FPGA design and verification · Verilog

Contents

OBJECTIVE 1
SUMMARY & INTRODUCTION 3
Summary 3
Introduction 3
PART A: Baud Rate Controller 4
Implementation Description and Key Code Sections 4
Calculation of the Relative Baud-Rate Error 4
Testbench Verification and Issues Encountered 5
PART B: UART Transmitter 6
Implementation Description and Key Code Sections 6
Testbench Verification and Issues Encountered 7
FPGA Implementation and Issues Encountered 8
PART C: UART Receiver 9
Implementation Description and Key Code Sections 9
Testbench Verification and Issues Encountered 10
FPGA Implementation and Issues Encountered 11
PART D: Integrated Transmitter–Receiver System 12
Implementation Description and Key Code Sections 12
Testbench Verification and Issues Encountered 12
CONCLUSIONS 14

SUMMARY & INTRODUCTION

Summary

This project involved implementing a complete UART transmitter-receiver system consisting of a Baud Rate Controller, a Transmitter, and a Receiver. The Baud Rate Controller is implemented as a multiplexer that generates a periodic output signal used by both the transmitter and the receiver to time data transmission and reception. When enabled, the Transmitter sends an 8-bit data sequence, while the Receiver accepts the data when enabled, following the Start Bit, Data Bits, Parity Bit, and Stop Bit states. The system operation was confirmed and verified through simulations and practical tests on the FPGA.

Introduction

The aim of this project was to implement a UART system incorporating a controller that generates sampling pulses, specifically defining the number of clock cycles required to advance the FSMs of both the Transmitter and the Receiver. The Transmitter was implemented as a six-state FSM that sequentially sends 8-bit data to the Receiver when enabled; otherwise, it remains inactive (OFF or IDLE). The Receiver consists of a five-state FSM and uses sampling to perform all necessary checks on the validity of the data received from the Transmitter and to detect possible errors at the beginning or end of a transmission.

PART A: Baud Rate Controller

Implementation Description and Key Code Sections

The Baud Rate Controller was implemented in a module named baud_controller. A combinational block determines the required number of clock cycles according to the selected baud_select value, while a sequential block uses a 16-bit counter to generate an enable signal once the specified number of cycles has elapsed.

The number of cycles is calculated using the following expression:

$$\frac{f_{\text{clk}}}{\text{baud_select} \times 16} = \frac{100\ \text{MHz}}{\text{baud_select} \times 16}$$

Baud-rate divider calculation for a 100 MHz clock.

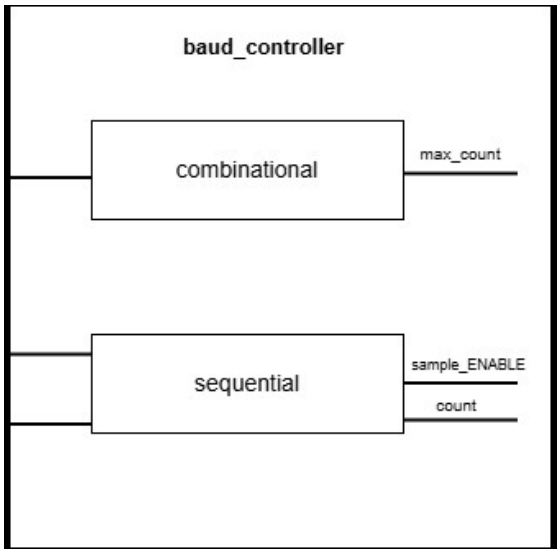
BAUD_SEL	Baud Rate *(bits/sec)*
0 0 0	300
0 0 1	1200
0 1 0	4800
0 1 1	9600
1 0 0	19200
1 0 1	38400
1 1 0	57600
1 1 1	115200

Baud-rate selection codes and supported bit rates.

Calculation of the Relative Baud-Rate Error

BAUD SELECT	BAD RATE ideal	multiplier	baudrate * 16	1/(baudrate*16)	* 10^8	round	ceiling	floor	error ceiling	error floor	best baud ratein cycles
000	300	16	4800	0,0002083333333	20833,33333	20833	20834	20833	0,000032	0,000016	20833
001	1200	16	19200	0,0000520833333	5208,333333	5208	5209	5208	0,000128	0,000064	5208
010	4800	16	76800	0,0000130208333	1302,083333	1302	1303	1302	0,000704	0,000064	1302
011	9600	16	153600	0,0000065104166667	651,0416667	651	652	651	0,001472	0,000064	651
100	19200	16	307200	0,0000032552083333	325,5208333	326	326	325	0,001472	0,001600	326
101	38400	16	614400	0,0000016276041667	162,7604167	163	163	162	0,001472	0,004672	163
110	57600	16	921600	0,0000010850694444	108,5069444	109	109	108	0,004544	0,004672	109
111	115200	16	1843200	0,0000005425347222	54,25347222	54	55	54	0,013760	0,004672	54

Relative baud-rate error calculations and selected count values.



Block diagram of the baud_controller module.

Testbench Verification and Issues Encountered

To simulate the controller, a testbench named merosA_tb was developed. It checks whether the controller operates correctly, specifically whether the sample_ENABLE signal is asserted at the correct times for different baud_select inputs.

As confirmed by thorough verification of all eight baud_select settings, the circuit operates correctly and the sample_ENABLE signal is asserted at the appropriate times.

PART B: UART Transmitter

Implementation Description and Key Code Sections

The Transmitter implementation includes a main Moore FSM module with six states:

- In the OFF state, the Transmitter is inactive. When Tx_EN and Tx_WR are asserted, it transitions to the IDLE state.

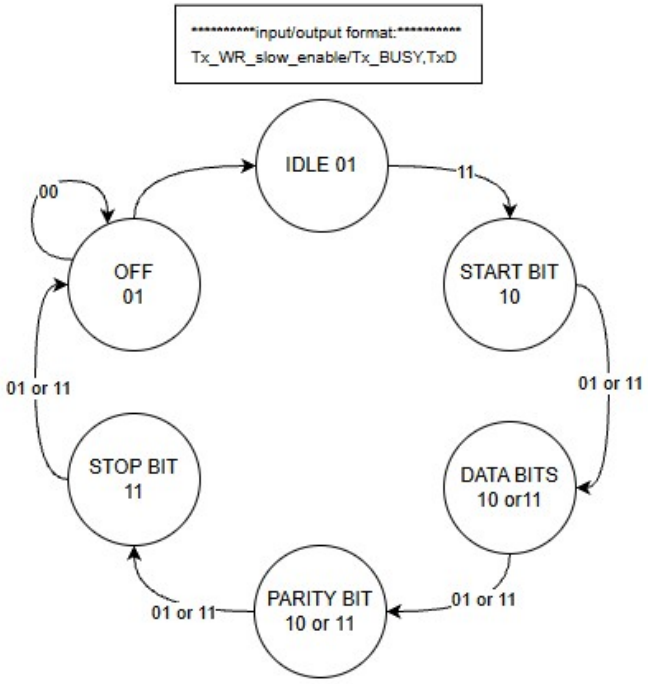
The Tx_WR signal is debounced using a Debouncer, which filters out any pulses shorter than 5 ms.

- In the IDLE state, the Transmitter remains inactive but available. Upon receiving slow_enable, it transitions to START_BIT.
- In the START_BIT state, TxD is driven low (0) to indicate the beginning of transmission. The Transmitter is marked as busy and waits for slow_enable before proceeding with the data.
- In the DATA_BITS state, the Transmitter sends one data bit at a time, with TxD taking the value of the current bit, either 0 or 1.
- In the PARITY_BIT state, the number of ones in the transmitted byte is evaluated. If the count is even, the parity bit is 0; if it is odd, the parity bit is 1. This is computed directly using an XOR operation. The Transmitter then moves to its final state, STOP_BIT.
- The STOP_BIT state marks the end of transmission by setting TxD to 1. While still marked as busy, the Transmitter then returns to the OFF state, where it becomes available again.

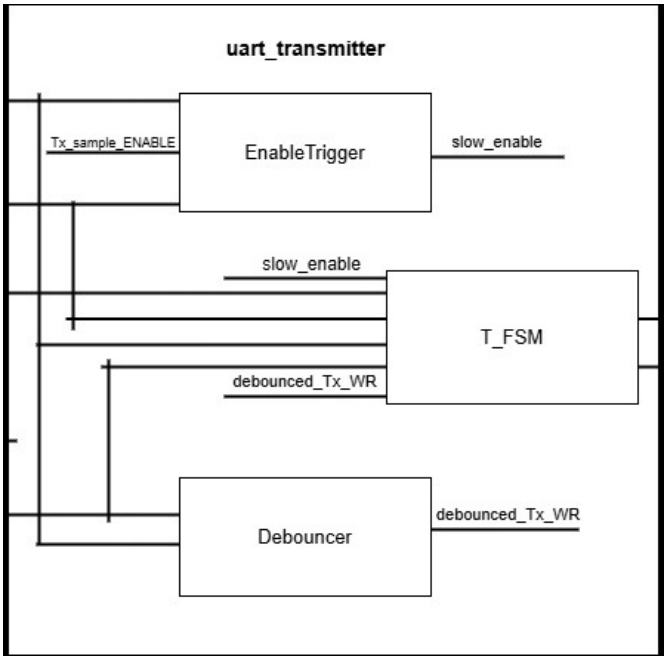
This implementation uses three always blocks: one sequential block to update the data-bit counter, another sequential block to manage state transitions, and a combinational block to define the behavior of each state and the Transmitter's availability. Transitions between active states occur only when slow_enable is asserted.

The slow_enable signal is generated by a separate module (EnableTrigger). This module receives sample_ENABLE from the Baud Rate Controller and uses a counter to assert slow_enable whenever the counter reaches 15, i.e., once every 16 assertions of sample_ENABLE.

All of these modules are connected in the top-level uart_transmitter module, which coordinates the selected baud rate, the generation of the appropriate enable signal, and the Transmitter FSM.



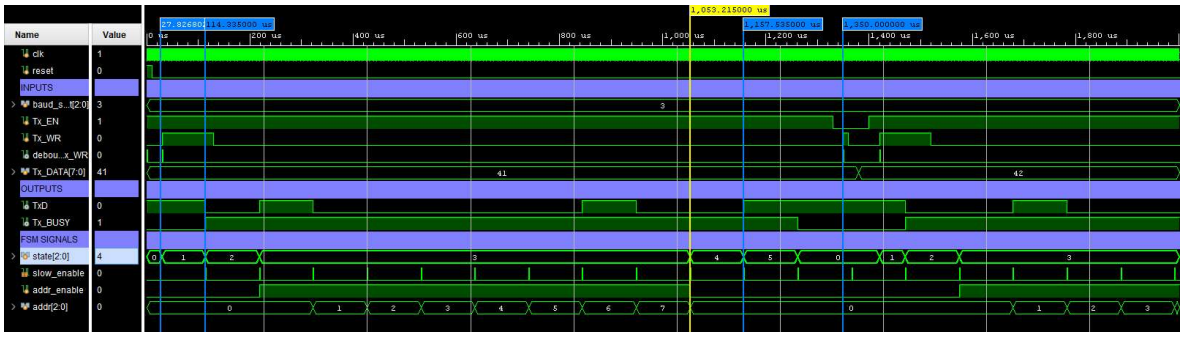
Six-state Moore FSM used in the UART transmitter.



UART transmitter architecture, including the debouncer and enable trigger.

Testbench Verification and Issues Encountered

To simulate the Transmitter comprehensively, a testbench was developed in which Tx_EN and Tx_WR are asserted at specific times to verify that they correctly control state transitions. A particular baud rate and several data values were also used to thoroughly verify that transmission takes place as intended.



Transmitter simulation waveform and control signals.

During the initial simulation, timing issues arose with Tx_WR and Tx_EN. These signals had to be synchronized with the FSM states so that their assertions would be detected and the required state transitions could occur. The issues were resolved by calculating the precise timing needed for coordination.

The tests confirmed that the system does not enter its first active state (START_BIT) when Tx_EN is 0 and does not transmit data when Tx_WR is inactive.

FPGA Implementation and Issues Encountered

During the initial hardware test of the Transmitter on the FPGA board, some characters received through the board were displayed incorrectly in Tera Term, appearing as unrecognized characters.

This occurred because the FSM transitioned directly from the inactive IDLE state to START_BIT when Tx_EN and Tx_WR were asserted, without checking slow_enable.

The issue was resolved by adding another state (OFF). The Transmitter now moves from OFF to IDLE when Tx_EN = Tx_WR = 1, and from IDLE to START_BIT only when slow_enable = 1.

PART C: UART Receiver

Implementation Description and Key Code Sections

The Receiver is built around a five-state Mealy FSM with the states IDLE, START, DATA, PARITY, and STOP, described below:

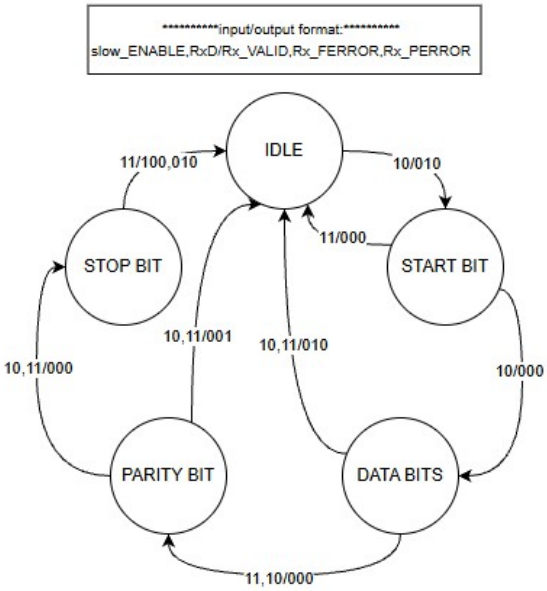
- In the IDLE state, the Receiver is inactive but ready. When Rx_EN becomes 1 and the RxD line is 0, the Receiver recognizes a possible start of transmission and enters START.
- In the START state, the RxD signal is sampled to determine whether it is valid. The transition to DATA occurs only when slow_enable is asserted, the sampling result is valid (ValidSample = 1), and the RxD sample has remained at 0. If sampling is invalid, FERROR is asserted and the Receiver returns to IDLE so that reception can restart.
- In the DATA state, the data is sampled four times and stored. When slow_enable becomes 1, addr reaches 7, and all eight samples have been validated, the FSM proceeds to PARITY.
- In the PARITY state, the parity bit is checked. If slow_enable is asserted, the sample is valid, and the parity computed from the previously received data matches RxD, the FSM proceeds to STOP. Otherwise, the parity error signal PERROR is asserted.
- In the STOP state, the stop bit is checked, and the Receiver returns to IDLE when slow_enable is asserted. If the sampled value is found to be 0 during this check, the framing error signal FERROR is asserted.

As in the Transmitter, the Receiver FSM module includes a sequential always block to handle state transitions and a combinational always block to define each state's outputs and behavior. Sampling is also implemented within the same module using a sequential always block sensitive to the clock and reset signals. A case statement evaluates a counter: samples are taken at counter values 4, 6, 8, and 10, and when the counter reaches 15, sample validity is assessed and the ValidSample and Sample signals are updated while slow_enable is asserted. In addition, the data-bit address counter and data storage are implemented as independent sequential circuits within the same FSM module.

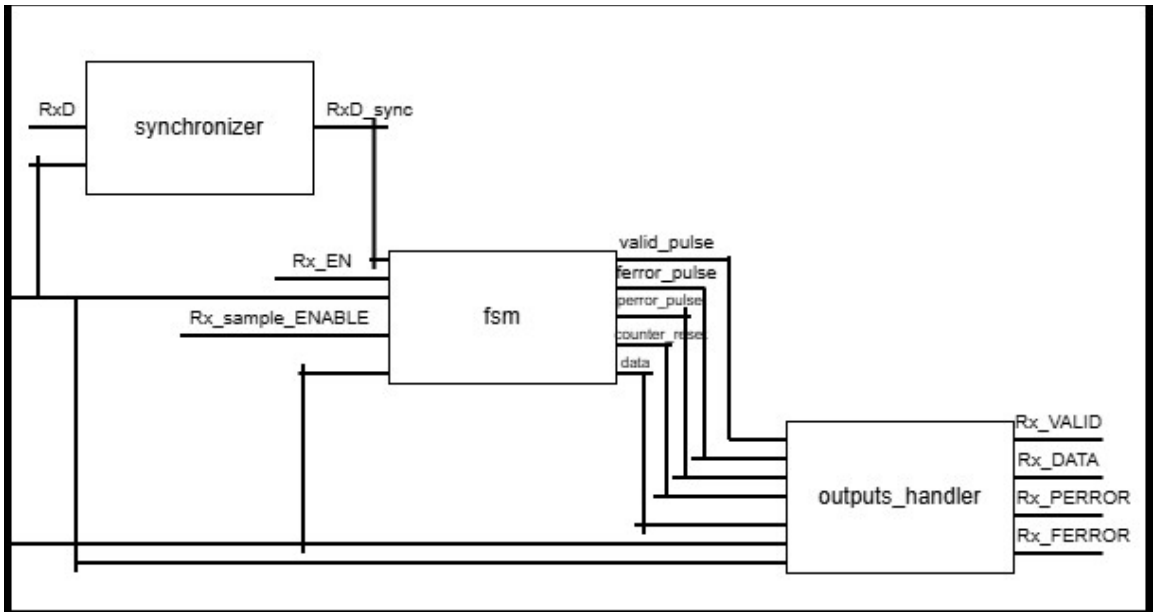
A separate outputs_handler module checks whether the FSM has generated Valid, Ferror, or Perror pulses. The corresponding output signals remain asserted until the next start bit so that they can be observed on the FPGA board.

An additional synchronizer module uses two flip-flops to synchronize the RxD signal to the clock.

All of these modules are connected in the top-level uart_receiver module.



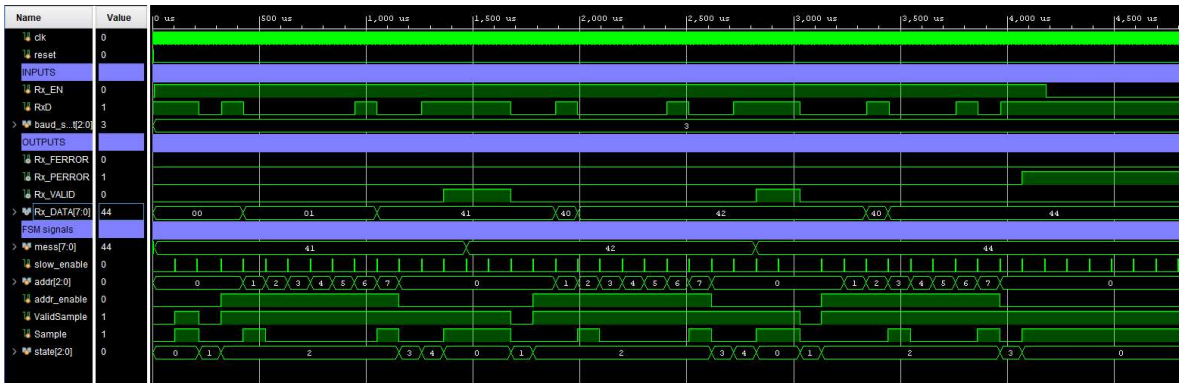
Five-state Mealy FSM used in the UART receiver.



UART receiver architecture with synchronizer, FSM, and outputs_handler.

Testbench Verification and Issues Encountered

To verify the Receiver's operation, a dedicated testbench was developed to monitor Rx_VALID, Rx_PERROR, Rx_FERROR, and Rx_DATA. The tests use an initial begin–end block to assign suitable values to the Rx_EN and Rx_D input signals. A variable named mess stores the message to be transmitted, while a for loop assigns successive values to RxD according to the message bits, starting with the least significant bit (LSB).



Receiver simulation waveform and status/error signals.

Inspection of the waveforms confirmed that the Receiver accepts data correctly only when Rx_EN = 1, that transitions between states occur under the intended conditions, and that the relevant error signals are asserted when required.

One issue was the absence of a signal informing the outputs_handler module that the next start bit had begun. Consequently, its outputs changed only when another error (Rx_FERROR or Rx_PERROR) or a valid-reception signal (Rx_VALID) was detected. This was resolved by making counter_reset an output of the FSM and using it in outputs_handler to clear the outputs at the beginning of the next start bit.

FPGA Implementation and Issues Encountered

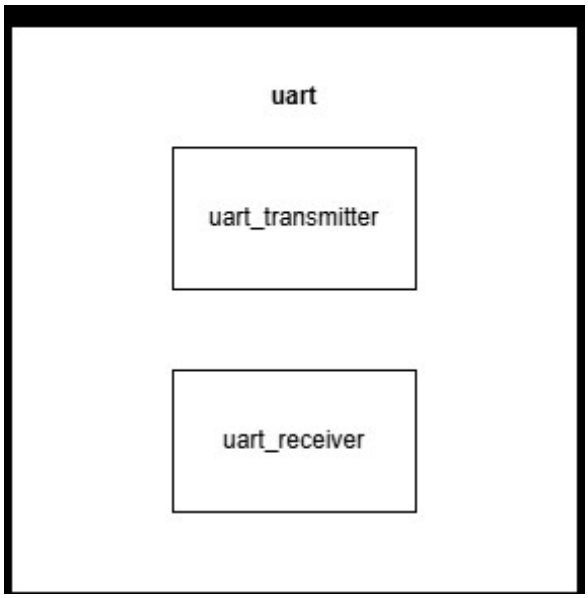
No problems were encountered when verifying the Receiver on the FPGA board with Tera Term. The LEDs representing the received data bits lit up according to the expected values, confirming correct frame reception. Likewise, Rx_VALID, Rx_FERROR, and Rx_PERROR behaved as intended, being asserted only for valid reception or their respective error conditions.

PART D: Integrated Transmitter–Receiver System

Implementation Description and Key Code Sections

In this part, the Transmitter and Receiver were connected over the UART channel, using the Rx_VALID and Tx_BUSY signals.

A top-level module named uart was implemented to instantiate the uart_transmitter and uart_receiver modules. In this system, the Transmitter sends data through TxD, while the Receiver samples that signal and performs the required checks.

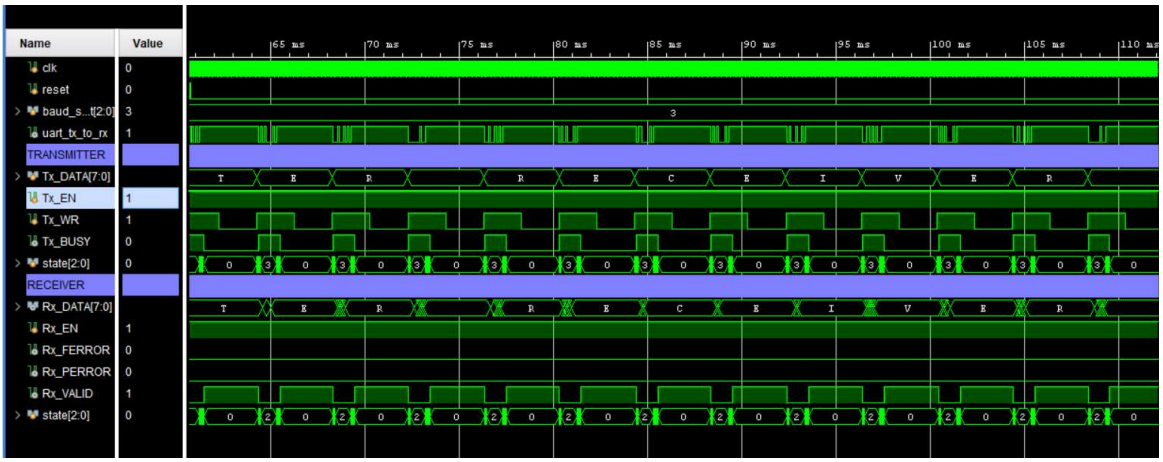


Top-level UART module connecting the transmitter and receiver.

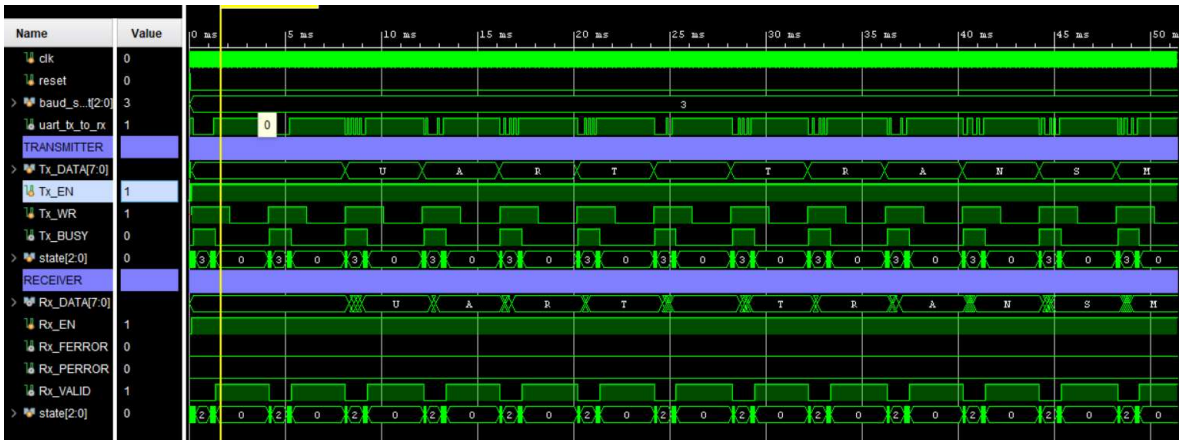
Testbench Verification and Issues Encountered

The system was simulated using a testbench named testbench_D, which instantiates the top-level uart module and assigns values to baud_select, Tx_EN, Tx_WR, Tx_DATA, and Rx_EN.

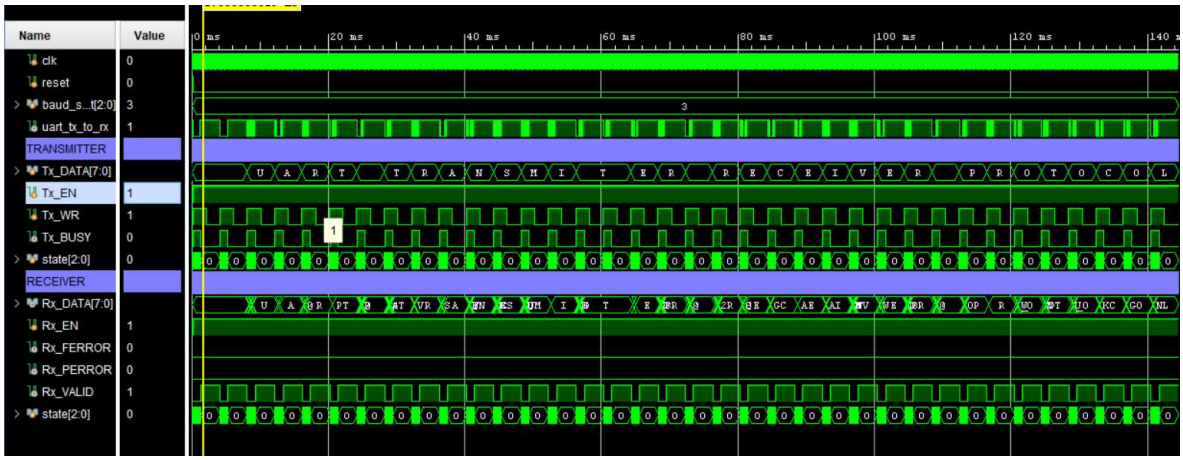
- All possible baud_select values were considered; however, for simplicity, 3'b011 was ultimately selected because its shorter timing made it easier to verify the results.
- Tx_EN and Tx_WR were initially set to 0. They were asserted after a suitable interval to allow the enable signal associated with baud_select to become active. Verification showed that data transmission and state transitions began only after these signals were asserted.
- The system was verified using four words. The results confirmed correct operation: while a byte was being transmitted, the next byte was not transmitted until the previous transmission had finished.



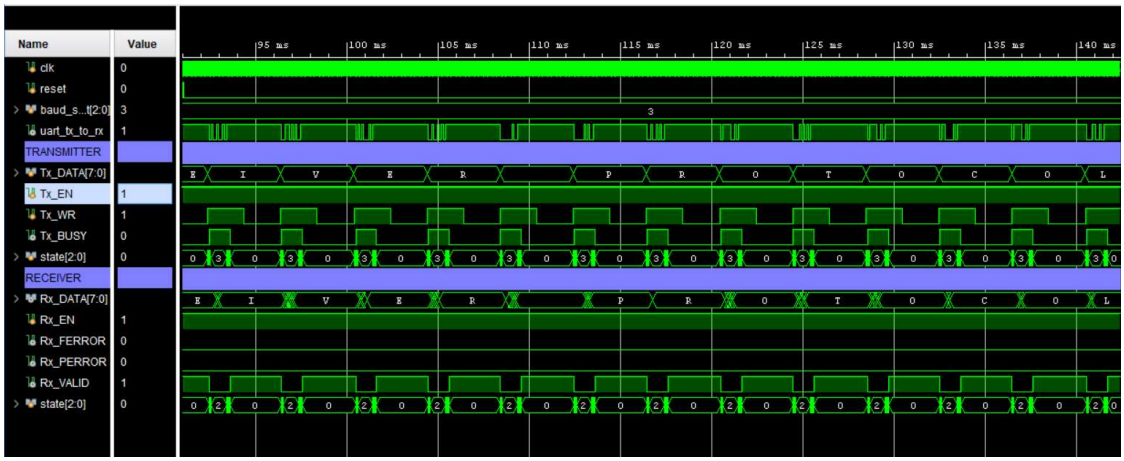
Full-system waveform: transmitter and receiver timing, overview 1.



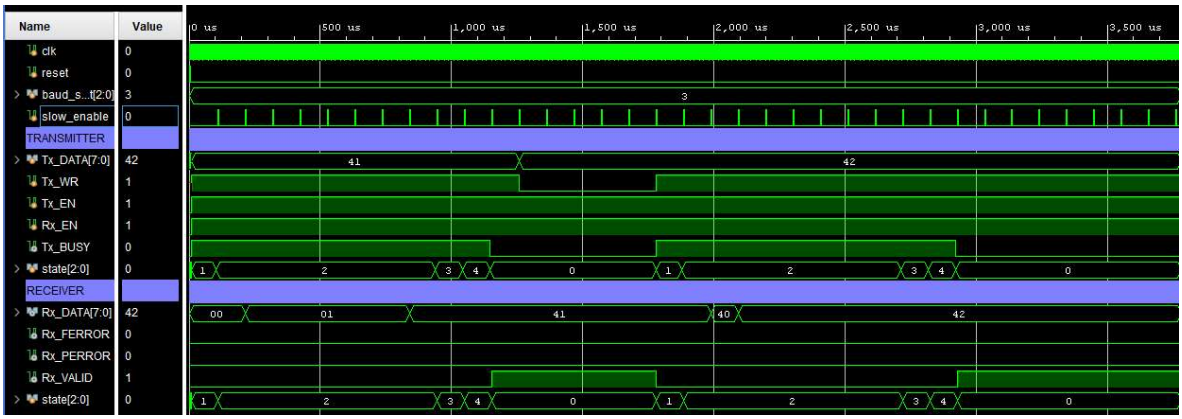
Full-system waveform: transmitter and receiver timing, overview 2.



Full-system waveform: transmitted and received data, segment 1.



Full-system waveform: transmitted and received data, segment 2.



Full-system waveform: detailed UART frame timing.

CONCLUSIONS

The project was completed successfully, with all necessary components of a UART transmitter-receiver system implemented and the challenges encountered during development resolved. The greatest challenge was synchronizing the Transmitter and Receiver (Part D) at the appropriate frequency and ensuring that the Receiver obtained the correct data. These difficulties were overcome by identifying errors in the individual modules through thorough simulation-based verification and practical testing on the FPGA board.